

CS-338 Intro to the Theory of Computation

- Lecture #13

➤ **Complexity Theory: NP Class**

- Prof Pietro S. Oliveto
Department of Computer Science and Engineering
Southern University of Science and Technology (SUSTech)
- `olivetop@sustech.edu.cn`
<https://faculty.sustech.edu.cn/olivetop>
- Office: Room 113

Overview

Last time:

- $\text{TIME}(t(n))$
- $P = \bigcup_k \text{TIME}(n^k)$
- $PATH \in P$
- $RELPRIME \in P$
- $CFL \in P$

Today: (Sipser §7.2 – §7.3)

- $\text{NTIME}(t(n))$
- NP
- P vs NP problem
- Polynomial-time reducibility

Quick Review

Defn: $\text{TIME}(t(n)) = \{B \mid \text{some deterministic 1-tape TM } M \text{ decides } B \text{ and } M \text{ runs in time } O(t(n))\}$

Defn: $P = \bigcup_k \text{TIME}(n^k)$
= polynomial time decidable languages

$PATH = \{\langle G, s, t \rangle \mid G \text{ is a directed graph with a path from } s \text{ to } t \}$

Theorem: $PATH \in P$

PATH and HAMPATH

Example: $HAMPATH = \{ \langle G, s, t \rangle \mid G \text{ is a directed graph with a path from } s \text{ to } t \text{ and the path goes exactly once through every node of } G \}$

Recall Theorem: $PATH \in P$

Called a Hamiltonian path

Question: $HAMPATH \in P$?

“On input $\langle G, s, t \rangle$

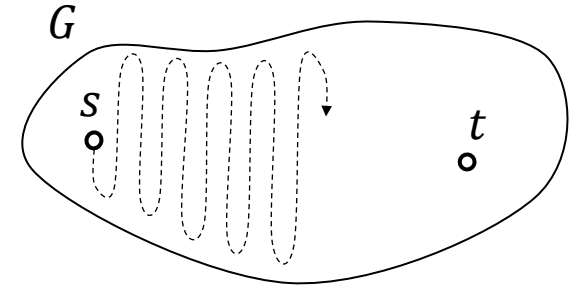
1. Let m be the number of nodes in G .
2. For each path of length m in G :
test if m is a Hamiltonian path from s to t .
Accept if yes.
3. Reject if all paths fail.”

May be $m! > 2^m$ paths of length m
so algorithm is exponential time
not polynomial time.

$HAMPATH \in P$? Unsolved Problem

We can avoid brute force search in many problems and obtain polynomial time solutions

Others we don't know



Polynomial verifiability

$HAMPATH = \{\langle G, s, t \rangle \mid G \text{ is a directed graph with a path from } s \text{ to } t \text{ that goes through every node of } G \text{ exactly once}\}$

$HAMPATH \in P$? Unsolved Problem

We can avoid brute force search in many problems and obtain polynomial time solutions

Others we don't know

Let's say I tell you this is a solution...

Then it's easy to verify in polynomial time whether this is true

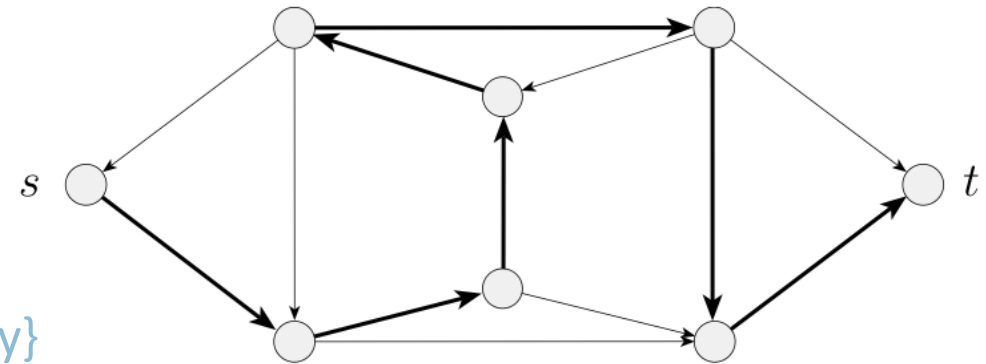
Defn: $COMPOSITES = \{x \mid x = yz \text{ for integers } y, z > 1, x \text{ in binary}\}$

If I give you a divisor y or z you can easily verify whether this is true

A **verifier** for a language A is an algorithm V , where

$$A = \{w \mid V \text{ accepts } \langle w, c \rangle \text{ for some string } c\}.$$

We measure the time of a verifier only in terms of the length of w , so a **polynomial time verifier** runs in polynomial time in the length of w . A language A is **polynomially verifiable** if it has a polynomial time verifier.



- c is called *certificate* or *proof*
- $HAMPATH$: for $\langle G, s, t \rangle$ c is a Hamiltonian path from s to t
- $COMPOSITES$: for x it is simply one of its divisors
- In both cases the verifier V can verify in polynomial time (in the length of w) that the input w is in the language given the certificate

Nondeterministic Complexity

In a nondeterministic TM (NTM) **decider**, all branches halt on all inputs.

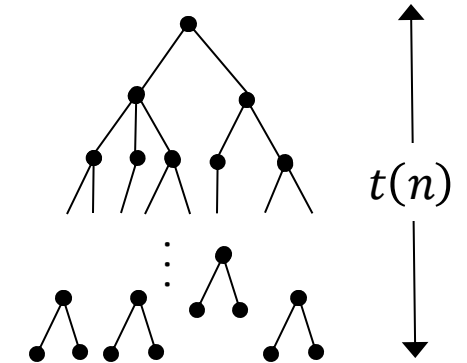
Defn: An NTM runs in time $t(n)$ if **all** branches halt within $t(n)$ steps on all inputs of length n .

Defn: $\text{NTIME}(t(n)) = \{B \mid \text{some 1-tape NTM decides } B \text{ and runs in time } O(t(n))\}$

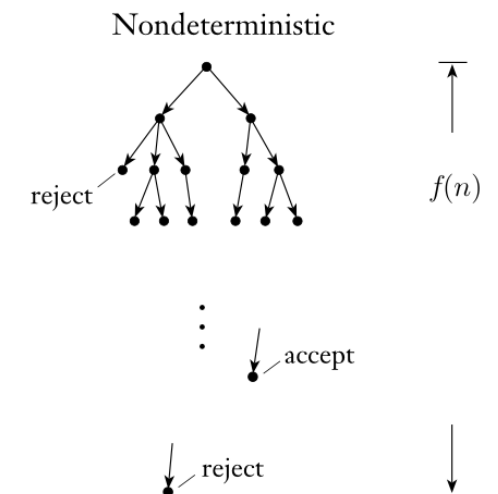
Defn: $\text{NP} = \bigcup_k \text{NTIME}(n^k)$
= nondeterministic polynomial time decidable languages

- Invariant for all reasonable nondeterministic models
- Corresponds to **easily verifiable** problems

Computation tree for NTM on input w .



all branches halt
within $t(n)$ steps



HAMPATH $\in$ NP

Theorem: HAMPATH $\in$ NP

Proof:

“On input $\langle G, s, t \rangle$ (Say G has m nodes.)

1. Nondeterministically write a sequence

$v_1, v_2, \dots, v_m$ of m nodes.

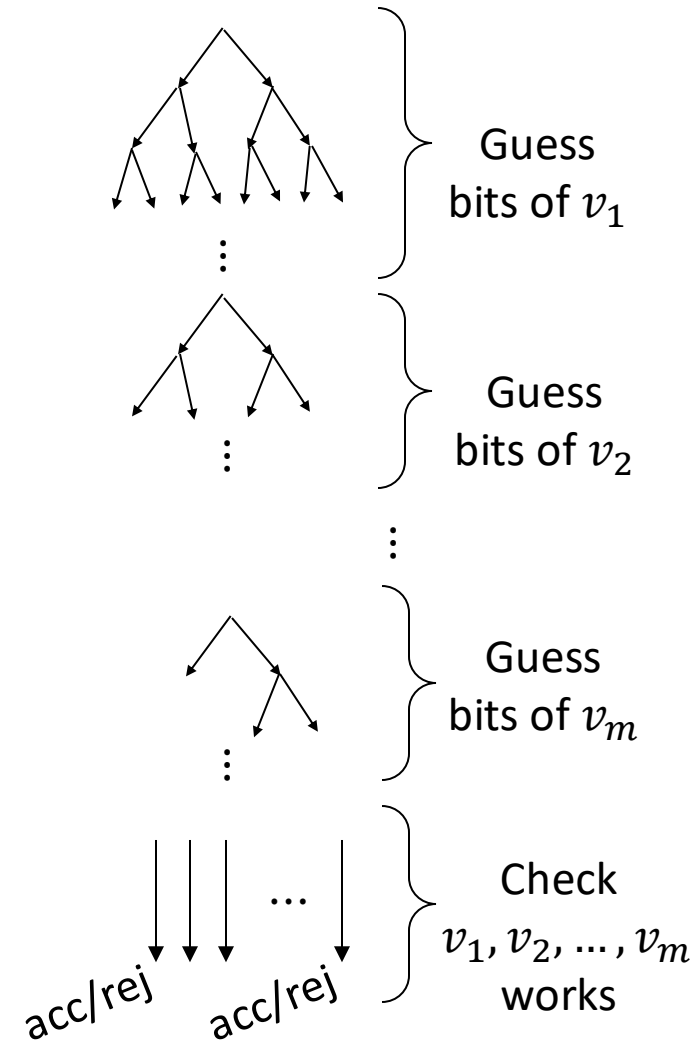
2. *Accept* if $v_1 = s$

$v_m = t$

each (v_i, v_{i+1}) is an edge
and no v_i repeats.

3. *Reject* if any condition fails.”

Computation of
M on $\langle G, s, t \rangle$



COMPOSITES $\in$ NP

Defn: $COMPOSITES = \{x \mid x \text{ is not prime and } x \text{ is written in binary}\}$
 $= \{x \mid x = yz \text{ for integers } y, z > 1, x \text{ in binary}\}$

Theorem: $COMPOSITES \in NP$

Proof: “On input x

1. Nondeterministically write y where $1 < y < x$.
2. *Accept* if y divides x with remainder 0.
Reject if not.”

Note: Using base 10 instead of base 2 wouldn’t matter because can convert in polynomial time.

Bad encoding: write number k in unary: $1^k = \overbrace{111 \cdots 1}^k$, exponentially longer.

Theorem (2002): $COMPOSITES \in P$

We won’t cover this proof.

NP languages have verifiers

Theorem: A language is in NP iff it is decided by a polynomial time verifier

Proof Idea: we show how to convert a poly time NTM into a poly time verifier and vice versa: the NTM simulates by guessing the certificate; the verifier simulates by using the accepting branch as certificate

Proof:

V verifier for A that runs in time $n^k \Rightarrow A$ is decided by a poly time NTM N

“On input w of length n :

1. Nondeterministically select c of length at most n^k .
2. Run V on $\langle w, c \rangle$
3. If V accepts, *Accept* otherwise *Reject*.”

A is decided by a poly time NTM $N \Rightarrow V$ verifier for A that runs in time n^k

“On input $\langle w, c \rangle$ where w and c are strings:

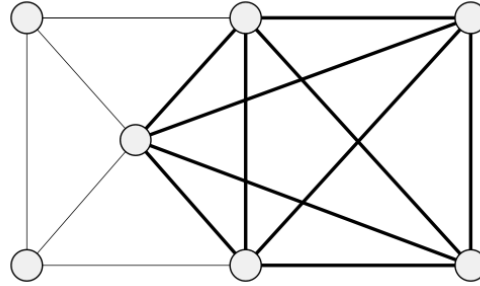
1. Simulate N on w using each symbol of c to decide which choice to make
2. If the branch of N 's computation accepts, *Accept* otherwise *Reject*.”

CLIQUE

A **clique** in an undirected graph is a subgraph wherein every two nodes are connected by an edge

A **k -clique** is a clique with k nodes

Eg., a 5-clique



Defn: $CLIQUE = \{ \langle G, k \rangle \mid G \text{ is an undirected graph with a } k\text{-clique} \}$

Theorem: $CLIQUE \in NP$

Proof: “On input $\langle G, k \rangle$ where G is a graph:

1. Nondeterministically select a subset c of k nodes of G
2. Test whether G contains all edges connecting the nodes in c
3. *Accept* if *yes*, otherwise *Reject*.”

Alternative proof: [using a verifier]

“On input $\langle \langle G, k \rangle, c \rangle$:

1. Test whether c is a subgraph with k nodes in G
2. Test whether G contains all edges connecting the nodes in c
3. *Accept* if both *yes*, otherwise *Reject*.”

SUBSET-SUM

SUBSET-SUM problem:

- We are given k numbers $x_1, x_2, \dots, x_k$ and a target number t .
- We want to know whether a subset of the k numbers sums up to t .

Defn: $SUBSET-SUM = \{ \langle S, t \rangle \mid S = \{x_1, \dots, x_k\} \text{ and for some } \{y_1, \dots, y_\ell\} \subseteq \{x_1, \dots, x_k\}, \text{ we have } \sum y_i = t \}$

Theorem: $SUBSET-SUM \in NP$

Proof: “On input $\langle S, t \rangle$:

1. Nondeterministically select a subset c of numbers in S .
2. Test whether the c numbers sum up to t .
3. *Accept* if *yes*, otherwise *Reject*.”

Alternative proof: [using a verifier]

“On input $\langle \langle S, t \rangle, c \rangle$:

1. Test whether c is a collection of numbers that sum up to t .
2. Test whether S contains all numbers in c
3. *Accept* if both *yes*, otherwise *Reject*.”

Intuition for P and NP

NP = All languages where can verify membership quickly

P = All languages where can test membership quickly

Examples of quickly verifying membership:

- *HAMPATH*: Give the Hamiltonian path.
- *COMPOSITES*: Give the factor.

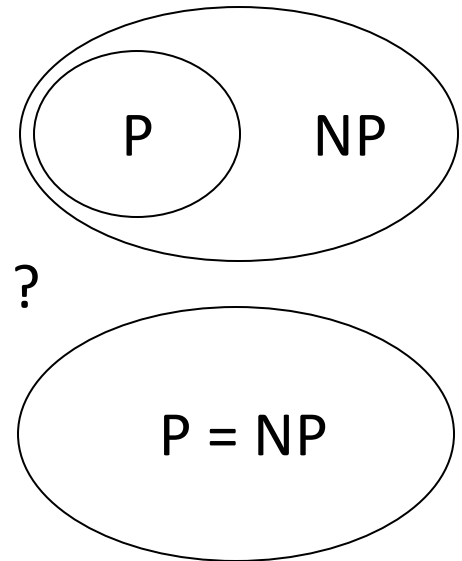
The Hamiltonian path and the factor are called ***short certificates*** of membership.

$P \subseteq NP$

Question: $P = NP$?

Conjecture: $P \neq NP$.

Hard to prove the conjecture because polynomial-time algorithms are powerful.



coNP

NP = All languages where can verify membership quickly

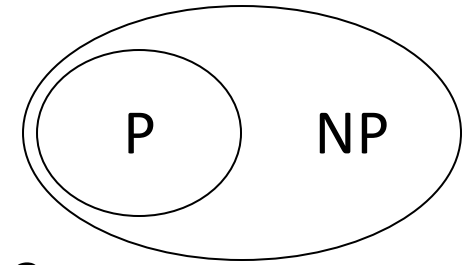
P = All languages where can test membership quickly

$\overline{HAMPATH}$, $\overline{CLIQUE}$, $\overline{SUBSET - SUM}$ are not obviously members of NP

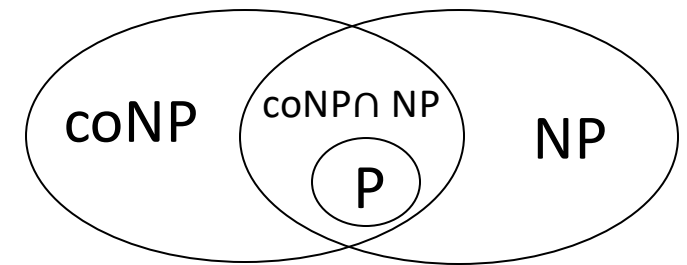
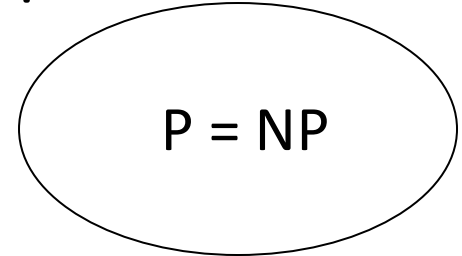
- Verifying that something is not present seems more difficult than verifying that it's present
- **coNP** is the class which contains the languages that are complements of languages in NP

coNP = All languages that have a short “no” certificate

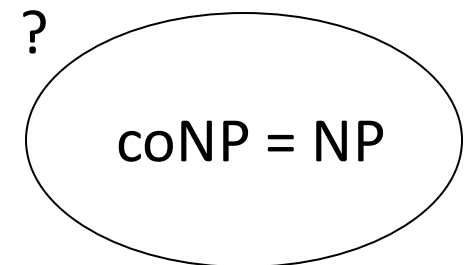
- Every “yes” instance of $A \in NP$ is a “no” instance of $\overline{A} \in coNP$
- We don't know whether coNP is different from NP
- Eg.,
 - PRIMES is in coNP
 - $x \notin PRIMES$: “no” certificate is simply a factorization of x .
Eg., $12 = 4 * 3$
 - $P \subseteq coNP \cap NP$
 - If $L \in P$, then the algorithm to recognize $\overline{L}$ simulate the TM for L and invert the answer
 - PRIMES is also in NP !!



?



?



NTM -> DTM conversion

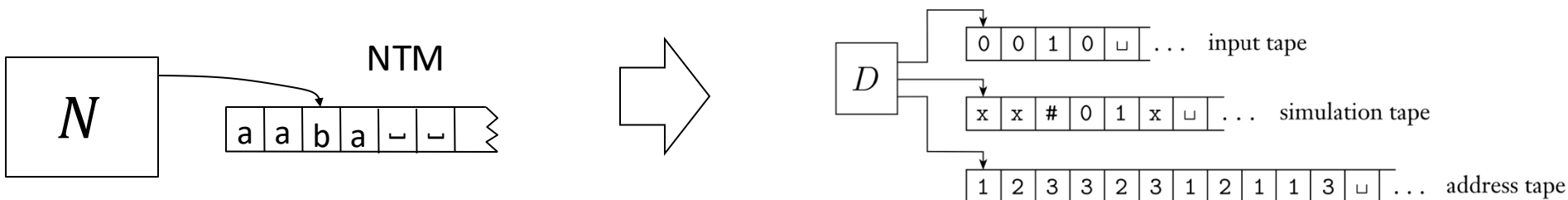
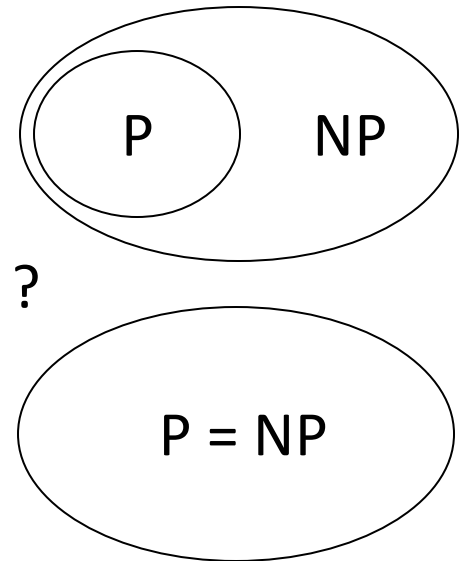
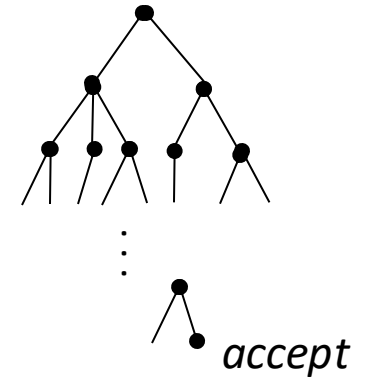
Theorem

Let $t(n) \geq n$. Then every $t(n)$ time nondeterministic single tape Turing machine has an equivalent $2^{O(t(n))}$ time deterministic single-tape Turing machine.

Proof

1. Construct the equivalent DTM (done).
2. Analyse the simulation time:
 - Each branch has computation time at most $t(n)$
 - Every node in the tree has at most b children (given by δ)
 - $\Rightarrow$ the total number of leaves in the tree is at most $b^{t(n)}$
 - Total number of nodes at most twice the max number of leaves: $O(b^{t(n)})$
 - The time to travel from the root to a node is at most $t(n)$
 - Altogether we get a running time of $O(t(n) * b^{t(n)}) = 2^{O(t(n))}$.
 - Converting from multi-tape to single tape at most squares the runtime $(2^{O(t(n))})^2 = 2^{O(2t(n))} = 2^{O(t(n))}$

□



Satisfiability Problem

Defn: A *Boolean formula* ϕ has Boolean variables (TRUE/FALSE values) and Boolean operations AND ($\wedge$), OR ($\vee$), and NOT ($\neg$).

Defn: ϕ is *satisfiable* if ϕ evaluates to TRUE for some assignment to its variables. Sometimes we use 1 for True and 0 for False.

Example: Let $\phi = (x \vee y) \wedge (\bar{x} \vee \bar{y})$ (Notation: $\bar{x}$ means $\neg x$)
Then ϕ is satisfiable ($x=1, y=0$)

Defn: $SAT = \{\langle \phi \rangle \mid \phi \text{ is a satisfiable Boolean formula}\}$

Theorem (Cook, Levin 1971): $SAT \in P \rightarrow P = NP$

Proof method: polynomial time (mapping) reducibility

Check-in 14.3

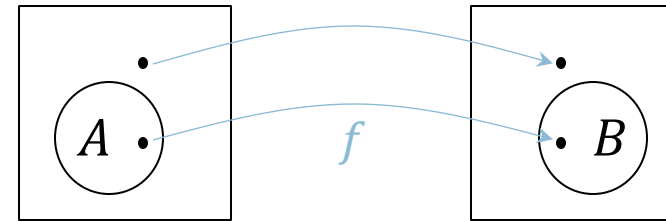
Is $SAT \in NP$?

- (a) Yes.
- (b) No.
- (c) I don't know.
- (d) No one knows.

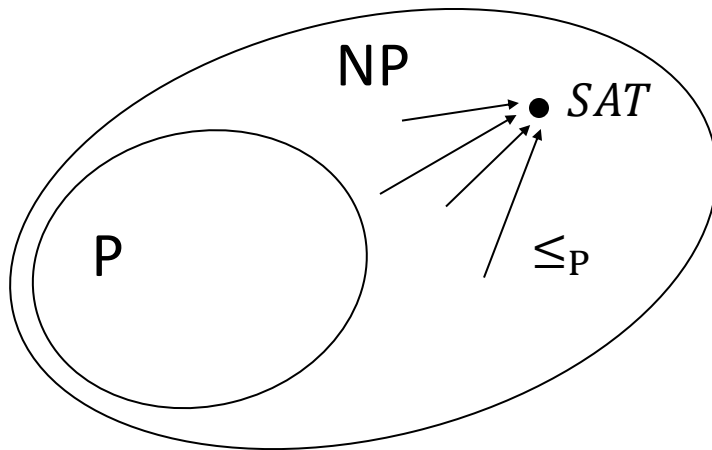
Polynomial Time Reducibility

Defn: A is polynomial time reducible to B ($A \leq_P B$) if $A \leq_m B$ by a reduction function that is computable in **polynomial time**.

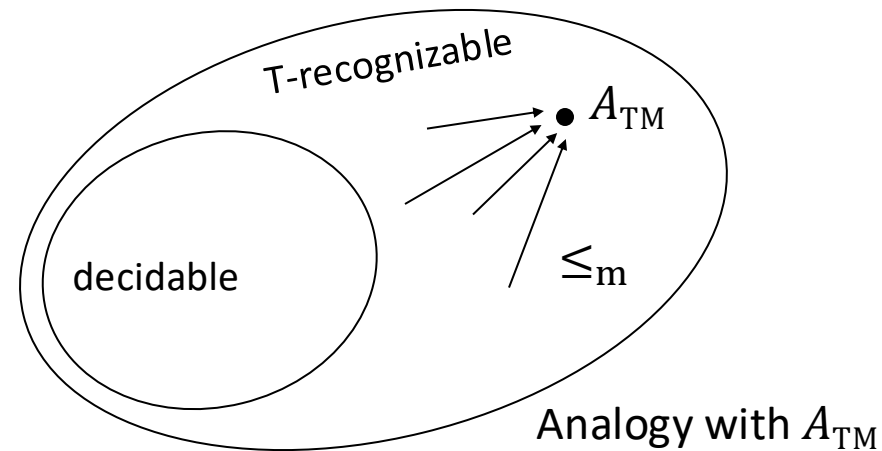
Theorem: If $A \leq_P B$ and $B \in P$ then $A \in P$.



f is computable in polynomial time



Idea to show $SAT \in P \rightarrow P = NP$



Analogy with A_{TM}

Quick review of today

1. $\text{NTIME}(t(n))$ and NP
2. *HAMPATH* and *COMPOSITES* $\in$ NP
3. *CLIQUE* and *SUBSET – SUM* $\in$ NP
4. P versus NP question
5. The Satisfiability Problem *SAT*
6. Polynomial time reducibility